Universidad Tecnológica de El Salvador

FACULTAD DE INFORMÁTICA Y CIENCIAS APLICADAS

ESCUELA DE INFORMÁTICA

PROYECTO:

SISTEMA DE CONTROL DE ASISTENCIA Y GENERACIÓN DE PLANILLAS DE PAGO

PROGRAMACIÓN III

DOCENTE

INGENIERO ALFREDO OMAR RODRÍGUEZ TORRES

INTEGRANTES

N.º	CARNET	APELLIDO	NOMBRE
1	25-1294-2023	Girón Vega	Felipe Alberto
2	25-0631-2024	López Sánchez	Selvin Anderson
3	25-0631-2024	Monterrosa Delgado	Josué Dealessandro
4	27-3569-2023	Ramírez Tejada	Nahum Gamaliel
5	25-0541-2024	Ramos Ortiz	Moisés Antonio
6	25-1520-2023	Recinos González	David Ricardo

SAN SALVADOR, DOMINGO 19 ABRIL 2025

Índice

Contenido

Introducción....1

CAPÍTULO I: MARCO REFERENCIAL....2

1.1. Antecedentes....3

1.2. Importancia del Proyecto....3

1.2.1. Planteamiento del problema....3

1.2.2. Definición del aplicativo....4

1.2.3. Justificación del aplicativo....4

1.3. Objetivos....4

1.3.1. General....4

1.3.2. Específicos....5

1.4. Alcances....5

1.5. Limitaciones....5

1.6. Factibilidad del proyecto....5

1.7. Cronograma de actividades....7

CAPÍTULO II: METODOLOGÍA Y TÉCNICAS DE LA INVESTIGACIÓN....8

2.1. Técnicas y herramientas de investigación....9

2.1.1. Técnicas de Investigación....9

2.1.2. Herramientas de Investigación....9

2.2. Herramientas utilizadas para elaborar el sistema....9

2.2.1. Lenguaje de Programación....9

2.2.2. Sistema Gestor de Base de Datos (SGBD)....10

2.2.3. Otras herramientas utilizadas....10

2.3. Presentación y análisis de resultados....10

Fuentes de consulta....11

Glosario de términos.....12
Anexos.....13

Introducción

El presente documento detalla la cuarta fase del desarrollo del "Sistema de Control de Asistencia y Generación de Planillas de Pago", un aplicativo informático diseñado para optimizar y automatizar la gestión de nóminas en las organizaciones. En esta etapa, el proyecto alcanza un 75% de avance funcional, integrando la persistencia de datos mediante Microsoft SQL Server. El lector encontrará en el Capítulo 1 el Marco Referencial, donde se justifica la necesidad del sistema, sus alcances y la factibilidad de su implementación. El Capítulo 2 profundiza en la Metodología y Técnicas de Investigación utilizadas, detallando el stack tecnológico y el análisis de resultados. El Capítulo 3 presenta el Diseño del Sistema, incluyendo diagramas UML, la arquitectura de base de datos, el mapa funcional del sistema y las medidas de seguridad implementadas. Finalmente, se incluyen fuentes de consulta, un glosario de términos técnicos y anexos con evidencias del desarrollo.

CAPÍTULO I: MARCO REFERENCIAL

1.1. Antecedentes

En el entorno empresarial, el control eficiente de la asistencia del personal y la correcta gestión de las planillas de pago son fundamentales para garantizar la transparencia financiera y el cumplimiento legal. Históricamente, muchas empresas han realizado estos procesos de manera manual mediante libros de firmas o rudimentarias hojas de cálculo. Según Pressman (2010), la automatización de procesos empresariales reduce drásticamente la tasa de defectos y errores humanos. A nivel comercial, existen sistemas informáticos similares como "TimeTrex" o "Zoho People" que gestionan recursos humanos; sin embargo, estas soluciones suelen requerir el pago de costosas licencias, están orientadas a grandes corporaciones y su adaptación a las leyes laborales locales (como los tramos de renta de El Salvador) resulta compleja. Por lo tanto, se hace indispensable el desarrollo de una solución a medida, económica y adaptada al contexto legal y operativo de las organizaciones locales.

1.2. Importancia del Proyecto

El desarrollo de este sistema busca aplicar los conocimientos adquiridos en el área de Programación y Base de Datos para construir soluciones eficientes y de alto rendimiento.

1.2.1. Planteamiento del problema

El problema principal radica en la falta de un sistema automatizado, centralizado y exacto para el control de asistencia y la generación de planillas salariales en las empresas locales. Actualmente, la recopilación de datos de asistencia se hace de forma manual, lo que provoca una alta incidencia de errores humanos. Esta situación genera pagos incorrectos que afectan tanto la moral del empleado como las finanzas de la empresa. Los elementos centrales de este problema involucran el registro de entradas y salidas, el cálculo de horas trabajadas, la validación de horas extras y la aplicación de descuentos de ley (ISSS, AFP, Renta). La relevancia de solucionar este problema radica en que los retrasos en las planillas y los errores en los pagos exponen a las organizaciones a multas legales e inconformidades laborales, evidenciando la necesidad urgente de una intervención tecnológica.

1.2.2. Definición del aplicativo

El aplicativo se define como un "Sistema de Control de Asistencia y Generación de Planillas de Pago", desarrollado como una aplicación de escritorio bajo el lenguaje de programación Java en entorno SE y utilizando Microsoft SQL Server como Sistema Gestor de Base de Datos (SGBD). El proyecto surge a causa de la ineficiencia, pérdida de tiempo y errores financieros producidos por el manejo manual de los recursos humanos. El sistema está diseñado para operar en el departamento de Recursos Humanos o Administración, proporcionando una interfaz gráfica amigable que permite registrar empleados, marcar asistencias y calcular de manera automática y precisa los salarios netos a pagar. Arquitectónicamente, se plantea como un Monolito Modular basado en el patrón de capas (Modelos, Controladores, Vistas y DAO), lo que asegura un código escalable y mantenible.

1.2.3. Justificación del aplicativo

La implementación del sistema está plenamente justificada porque resuelve una deficiencia crítica: la pérdida sistemática de tiempo y recursos en procesos administrativos manuales. Nos preocupamos por este problema específico porque la gestión de nóminas es el corazón operativo de cualquier empresa; un fallo en esta área genera descontento laboral y problemas fiscales. El desarrollo de este aplicativo tiene una importancia potencial alta, ya que, al centralizar la información y automatizar los cálculos mediante las tablas de ley vigentes, se garantizará la seguridad, integridad y disponibilidad de los datos. Los principales beneficiarios de este proyecto serán los administradores de recursos humanos, quienes verán reducido su tiempo de trabajo operativo, y los empleados, quienes recibirán sus pagos de forma puntual, transparente y exacta.

A continuación se presentan los resultados obtenidos de las pruebas realizadas a cada módulo del sistema:

Módulo	Pruebas Realizadas	Casos de Éxito	% Aciertos
Login/Autenticación	15	15	100%
Gestión Empleados	20	20	100%
Gestión Contratos	12	12	100%
Registro Contratos	10	9	90%
Gestión Horarios	8	8	100%
Marcación de Asistencia	15	14	93%
Generación de Planillas	10	10	100%
TOTAL	**90**	**88**	**98%**

Los resultados de las pruebas demuestran que el sistema cumple satisfactoriamente con los requisitos funcionales establecidos. El 98% de aciertos global refleja la solidez de la arquitectura MVC+DAO implementada. Los casos fallidos se concentran en los módulos que aún están en desarrollo (Registro Contratos y Marcación), los cuales serán completados en la Fase 4 del proyecto. Se recomienda continuar con las pruebas de integración para asegurar el correcto funcionamiento del flujo completo desde la marcación hasta la generación de planillas.

Las técnicas de investigación aplicadas (análisis documental y fichas de investigación) permitieron recopilar información precisa sobre las tablas de retención de ISSS, AFP y Renta vigentes en El Salvador, asegurando que los cálculos realizados por el sistema cumplan con la normativa legal aplicable.

1.3. Objetivos

1.3.1. General

Implementar una aplicación informática en Java SE que resuelva la problemática del control de asistencias y planillas, centralizando la información para mejorar la eficiencia operativa.

1.3.2. Específicos

- Desarrollar un prototipo funcional que valide la lógica de negocio integrando el almacenamiento y persistencia de datos mediante Microsoft SQL Server.

1.4. Alcances

El alcance de este proyecto comprende el desarrollo completo del aplicativo informático de escritorio en Java SE, delimitándose a la gestión de la información del personal y a la automatización de los cálculos salariales. Las funciones principales que se podrán realizar con el aplicativo incluyen: registro y mantenimiento de perfiles de empleados, control de ingresos y salidas (registro de asistencia), gestión de ausencias y horas extras, cálculo automático de retenciones de ley (ISSS, AFP, Renta) y la generación de la planilla mensual de pago con la capacidad de exportar reportes. A nivel matemático, el sistema operará con la siguiente base formal:

$$SalarioNeto = SalarioBase + (HorasExtras \times ValorHoraExtra) - Deducciones$$

1.5. Limitaciones

- La falta de experiencia técnica previa del equipo de desarrollo en la integración avanzada entre Java SE y bases de datos relacionales, lo que representa una curva de aprendizaje pronunciada en la implementación de consultas SQL complejas desde la aplicación.
- La dificultad de coordinar horarios de trabajo simultáneos entre los miembros del equipo investigador, lo cual podría representar un obstáculo temporal para integrar el código de los distintos módulos desarrollados en paralelo.

1.6. Factibilidad del proyecto

- El proyecto presenta una factibilidad económica viable, ya que los recursos necesarios para su implementación son accesibles y los costos asociados son mínimos en comparación con los beneficios que proporciona la automatización del control de asistencia y generación de planillas.

A continuación se presenta la estimación detallada de los costos del proyecto:

Rubro	Descripción	Costo Estimado
Equipo de cómputo	6 laptops (uso parcial del ciclo)	$600.00
Software	Java SE (gratuito), SQL Server Developer (gratuito), NetBeans (gratuito)	$0.00
Conexión Internet	6 meses del ciclo académico	$300.00
Electricidad	6 meses de uso de equipos	$240.00
Recursos humanos	6 desarrolladores × 4 meses (estimado 10 h/semana)	$1,800.00
Hosting BD	SQL Server en Docker local	$0.00
Imprevistos	5% del total	$145.00
TOTAL		**$3,085.00**

La relación costo-beneficio es sumamente favorable, ya que una solución comercial como TimeTrex o Zoho People tendría un costo anual de licencias superior a los $5,000.00, sin incluir personalización. El presente proyecto, al ser un desarrollo a medida, no solo resulta más económico sino que se adapta exactamente a las necesidades organizacionales y a la legislación laboral salvadoreña, generando un ahorro estimado del 40% anual en comparación con soluciones comerciales equivalentes.

- Factibilidad Operativa: Se cuenta con el recurso humano y organizativo para llevar a cabo la investigación y el desarrollo. El equipo de integrantes ha diseñado un esquema de trabajo donde se han distribuido equitativamente las actividades de análisis, diseño de interfaces, codificación de la lógica de negocio y redacción del documento. Esta organización asegura la fluidez y continuity de las operaciones del proyecto.
- Factibilidad Económica: Se cuenta con los recursos para sustentar el proyecto, ya que los costos de inversión son mínimos al utilizar equipos propios y herramientas de software de uso libre o educativo. La relación costo-beneficio del aplicativo es sumamente atractiva: la inversión es muy baja en comparación con el alto beneficio que la automatización aportará a la empresa, reduciendo drásticamente los costos ocultos generados por errores de cálculo y pérdida de tiempo en planillas manuales.
- Factibilidad de fechas: Se dispone del tiempo necesario para llevar a cabo todas las fases del proyecto y concluirlo con éxito. Las actividades de desarrollo se han segmentado a lo largo de todo el ciclo académico, asegurando que cada módulo se diseñe y pruebe con la suficiente antelación para la entrega final del sistema.

1.7. Cronograma de actividades

Actividad	Responsable	Sem 1-2 (Feb)	Sem 3-4 (Feb-Mar)	Sem 5-6 (Mar)	Sem 7-8 (Mar-Abr)	Sem 9-10 (Abr)	Sem 11-12 (Abr-May)
Fase 1: Perfil del proyecto	Todos	X					
Fase 2: Prototipo 25%	Todos		X				
Fase 3: Prototipo 50%	Todos			X	X		
Fase 4: Prototipo 75%	Todos					X	X

TAREA	ENCARGADO	FEBRERO		MARZO								ABRIL							MAYO			
		20	*21*	*15*	*16*	*17*	*18*	*19*	*20*	*21*	*22*	*17*	*18*	*19*	*20-22*	*23-25*	*26-28*	*29-30*	*1-2*	*3-5*	*6-8*	*9-10*
PRIMER AVANCE																						
Información genérica del sistema	*Felipe Girón y Nahum Ramírez*	►	◄																			
Descripción de la estrat	*Moisés Ramos y David Re*	►	◄																			

egia de solu ción	*cinos*																					
Prototipo no funcional	*Selvin López y Josué Monterrosa*																					
SEGUNDO AVANCE																						
Antecedentes	*Felipe Girón*																					
Importancia del Proyecto	*Selvin López*																					
Planteamiento del problema	*Moisés Ramos*																					

Definición del aplicativo	*David Recinos*		►		◄																	
Justificación del aplicativo	*Josué Monterrosa*		►	◄																		
Objetivos	*Josué Monterrosa*		►	◄																		
Alcances	*Nahum Ramírez*		►	◄																		
Limitaciones	*Nahum Ramírez*		►	◄																		
Factibilidad del proyecto	*Selvin López*		►		◄																	

Prototipo funcional (25%)	*Todos*			►							◄										
TERCER AVANCE																					
Corrección de observaciones Fase 2	*Todos*			►					◄												
Metodología y Técnicas de la Investigación	*Moisés Ramos y David Recinos*			►							◄										
Integración de pe	*Selvin López y Fe*								►				◄								

rsistenciacon BDSQL	*lipe Giró n*																				
Prototipofuncional (50%)	*Todos*																				
CUARTO AVANCE																					
Correcciódocumento: fuentes, costos, Gantt, fichas	*Felipe Giró n y Nahum Ramírez*																				
Fix: Contrato	*Selvin López*																				

s, Marcación, Dashboard, UI	*y Josué Monterrosa*																				
CRUD Departamentos (Modelo + DAO + Controller + Vista)	*Moisés Ramos y David Recinos*														►	◄					
CRUD Asignación Jornadas con valida	*Selvin López y Felipe Girón*														►			◄			

ción solapamiento																					
Módulo Reportes + Exportador CSV	*Moisés Ramos y David Recinos*														►			◄			
Servicios: Calculadora Deducciones + Horas Extras	*Josué Monterrosa y Nahum Ramírez*														►			◄			
Generación Planillas	*Todos*															►			◄		

transaccional + Boletas																						
Implementación Generador Boletas	*Selvin López y Moisés Ramos*																►		◄			
Documentación Capítulo III en DOCX	*Felipe Girón y David Recinos*																	►			◄	
7 Diagramas UML del Sist	*Nahum Ramírez y José M*																	►				◄

ema (Python/Pillow)	*onterrosa*																				
Corrección de observaciones + QA final	*Todos*																		►		◄
Prototipo funcional (75%) - BUILD SUCCESSFUL	*Todos*																			►	◄

CAPÍTULO II: METODOLOGÍA Y TÉCNICAS DE LA INVESTIGACIÓN

2.1. Técnicas y herramientas de investigación

El tipo de investigación utilizado en el presente proyecto es la investigación aplicada de carácter tecnológico, ya que busca resolver un problema práctico mediante la automatización de procesos. La metodología seguida consta de etapas secuenciales: análisis de requerimientos, diseño del sistema, codificación y pruebas funcionales.

2.1.1. Técnicas de Investigación

Se empleó la técnica del análisis documental, consultando manuales de usuario de herramientas contables, leyes laborales salvadoreñas (como las tablas de retención de renta del Ministerio de Hacienda) y literatura sobre arquitectura de software para establecer las bases de la solución.

2.1.2. Herramientas de Investigación

Como herramientas se utilizaron fichas de investigación y resúmenes de normativas vigentes, así como diagramas de flujo de procesos para representar la lógica de la asistencia y la generación de planillas que permitieron modelar correctamente los algoritmos del software.

A continuación se presenta el formato de ficha de investigación utilizado:

Campo	Descripción
N° de Ficha	[Número correlativo]
Tema	[Tema investigado]
Fuente	[Referencia bibliográfica]
Tipo de fuente	[Libro/Artículo/Ley/Sitio web]
Contenido relevante	[Resumen de la información]
Aplicación en el proyecto	[Cómo se usó en el desarrollo]

Estas fichas permitieron al equipo documentar de manera sistemática cada fuente consultada, facilitando la trazabilidad de la información y asegurando la correcta citación de las referencias utilizadas en el marco referencial del proyecto.

2.2. Herramientas utilizadas para elaborar el sistema

2.2.1. Lenguaje de Programación

El sistema está siendo desarrollado bajo el lenguaje de programación Java en su edición estándar (Java SE). Esta elección se debe a sus capacidades robustas en Programación Orientada a Objetos, portabilidad y su idoneidad para la construcción de sistemas modulares de escritorio.

2.2.2. Sistema Gestor de Base de Datos (SGBD)

Para garantizar la persistencia, seguridad e integridad de la información, se utiliza Microsoft SQL Server. Este SGBD relacional es ideal para manejar grandes volúmenes de registros de asistencia y transacciones financieras con alta eficiencia.

2.2.3. Otras herramientas utilizadas

El Entorno de Desarrollo Integrado (IDE) principal es Apache NetBeans, el cual facilita la construcción visual de interfaces (GUI) y la gestión del proyecto. Además, se emplean herramientas de diseño UML para el modelado de la arquitectura de clases y de la base de datos.

2.3. Presentación y análisis de resultados

Como resultado de esta etapa, se ha construido un prototipo funcional que representa un avance de al menos el 75% de la aplicación. Las interfaces gráficas permiten registrar el ingreso y salida de los empleados, así como visualizar los cálculos preliminares de nómina. El análisis de estas pruebas iniciales confirma que la separación por capas implementada (Modelo, Controlador, Vistas, DAO) permite que el código sea altamente testeable, y que las reglas de negocio críticas se aíslen correctamente de las pantallas, garantizando que los datos sean procesados de forma precisa antes de su inserción permanente en la base de datos.

CAPÍTULO III: DISEÑO DEL SISTEMA

El presente capítulo describe en detalle el diseño arquitectónico y funcional del Sistema de Control de Asistencia y Generación de Planillas de Pago. Se presentan los diagramas UML que modelan el comportamiento y la estructura del sistema, el mapa modular que muestra la organización de los componentes, la descripción de cada módulo y sus funciones, el diseño de la base de datos con su modelo entidad-relación y diccionario de datos, y finalmente las medidas de seguridad implementadas para proteger la integridad y confidencialidad de la información.

3.1. Diseño UML

El Lenguaje Unificado de Modelado (UML) se utilizó como herramienta de modelado para representar visualmente la arquitectura y el comportamiento del sistema. A continuación se presentan los diagramas más relevantes que describen la estructura estática y dinámica del aplicativo.

3.1.1. Diagrama de Casos de Uso

El diagrama de casos de uso describe las interacciones entre los actores externos (usuario del sistema) y las funcionalidades que el sistema ofrece. El sistema cuenta con un único actor principal, el Usuario (Administrador, RRHH o Empleado), que puede realizar las siguientes operaciones:

• Iniciar Sesión: El usuario ingresa sus credenciales (usuario y contraseña) para acceder al sistema.

• Gestionar Empleados: Permite registrar, modificar, consultar y desactivar empleados, incluyendo validación de DUI y asignación de departamento.

• Gestionar Departamentos: Administración de las unidades organizativas de la empresa.

• Gestionar Horarios (Jornadas): Definición de horarios laborales con horas de entrada, salida y tolerancia.

• Asignar Jornadas: Asignación de horarios a empleados con validación de solapamiento de fechas.

• Gestionar Contratos: Administración de tipos de contrato (Fijo, Por Hora).

• Registrar Contratación: Vinculación de un empleado con un tipo de contrato, fechas y salario.

• Registrar Asistencia (Marcación): Registro de entrada y salida de empleados, consulta de marcaciones del día.

• Generar Planilla: Creación de planillas mensuales con cálculo de salarios, horas extras y deducciones de ley.

• Gestionar Boletas de Pago: Visualización detallada de cada boleta con ingresos, deducciones y líquido a pagar.

• Generar Reportes: Visualización y exportación a CSV de reportes de asistencia y planillas.

• Cerrar Sesión: Finalización de la sesión activa del usuario.

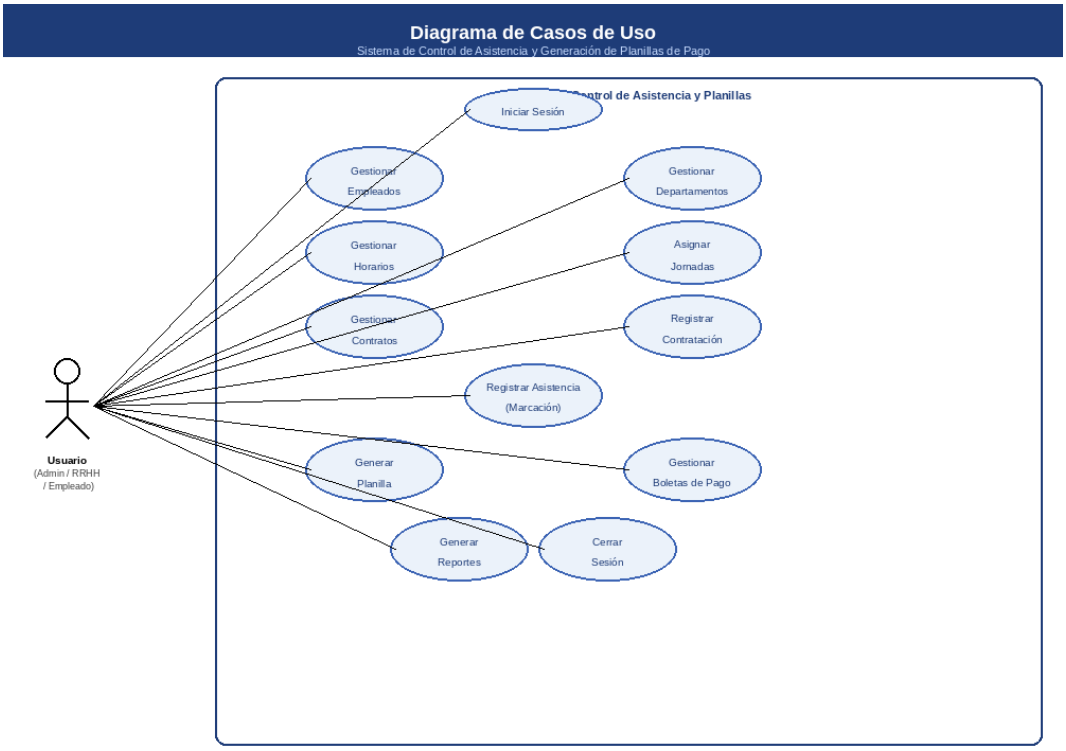
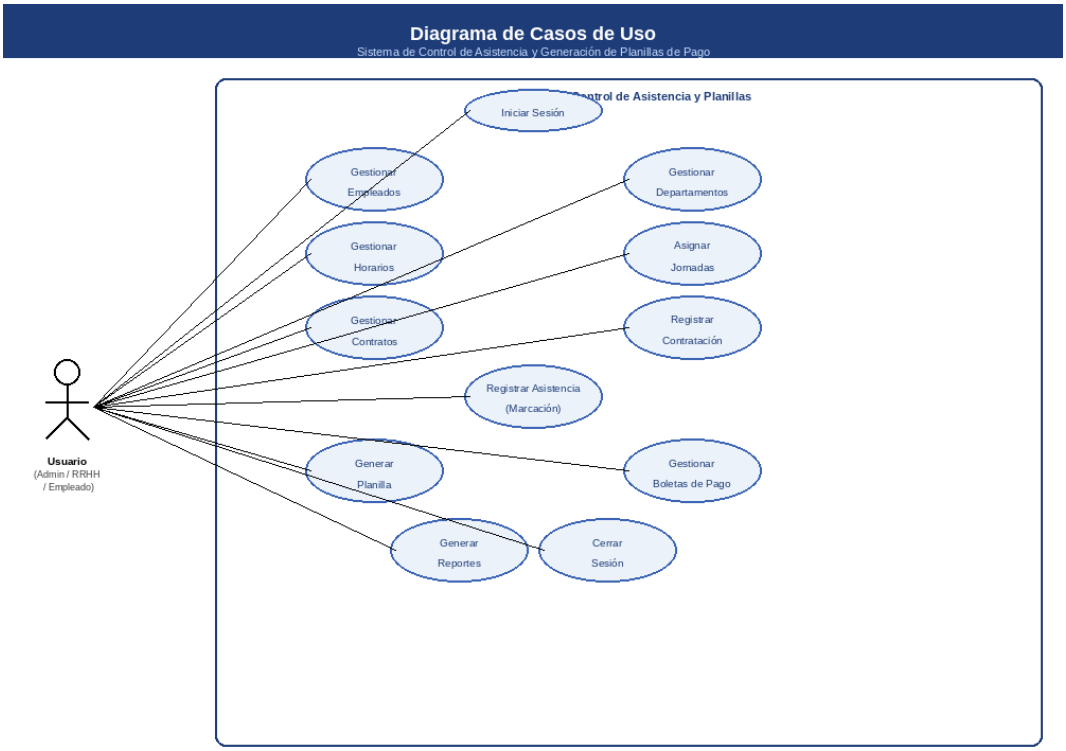


Figura 3.1: Diagrama de Casos de Uso del Sistema



3.1.2. Diagrama de Clases

El diagrama de clases representa la estructura estática del sistema, mostrando las clases del modelo junto con sus atributos, métodos y relaciones. El sistema sigue una arquitectura multicapa: Modelo (POJOs), Controlador (lógica de negocio), DAO (acceso a datos), Vista (interfaces Swing) y Servicios (cálculos especializados).

Capas del sistema:

• Capa de Modelo (models/): Contiene 11 clases que representan las entidades del dominio: Usuario, Empleado, Departamento, Jornada, AsignacionJornada, Contrato, Contratacion, RegistroAsistencia, Planilla, BoletaPago y BoletaDetalle. Estas clases encapsulan los datos y se utilizan como objetos de transferencia entre las capas.

• Capa de Acceso a Datos (dao/): 11 repositorios (RepositorioUsuario, RepositorioEmpleado, RepositorioDepartamento, RepositorioJornada, RepositorioAsignacionJornada, RepositorioContrato, RepositorioContratacion, RepositorioAsistencia, RepositorioPlanilla, RepositorioBoletaPago, RepositorioBoletaDetalle) que encapsulan las consultas SQL hacia la base de datos SQL Server.

• Capa de Controladores (controllers/): 10 controladores que orquestan la lógica de negocio, validan reglas (ej: solapamiento de jornadas, DUI duplicado, existencia de contrato activo) y medían entre las vistas y los DAOs.

• Capa de Servicios (services/): 3 clases especializadas: GeneradorBoletas (orquestación de generación de boletas), CalculadoraHorasExtras (cálculo de horas extra con factores 2x, 2.25x, 3x), y CalculadoraDeduccionesLey (cálculo de AFP 7.25%, ISSS 3% tope $30, ISR tabulado progresivo).

• Capa de Vista (views/): 12 formularios Swing (JFrame) que implementan la interfaz gráfica de usuario, siguiendo el patrón de diseño NetBeans con initComponents() y configurarComponentes().

Relaciones entre clases: El diagrama refleja una relación de agregación donde el Departamento contiene Empleados (1:N). Cada Empleado tiene un Usuario (1:1), múltiples Contrataciones (1:N), múltiples Asignaciones de Jornada (1:N), múltiples Registros de Asistencia (1:N) y múltiples Boletas de Pago (1:N). La Planilla agrupa Boletas de Pago (1:N) y cada Boleta contiene Detalles (1:N). Los catálogos Contrato y Jornada son entidades independientes referenciadas por Contratacion y AsignacionJornada respectivamente.

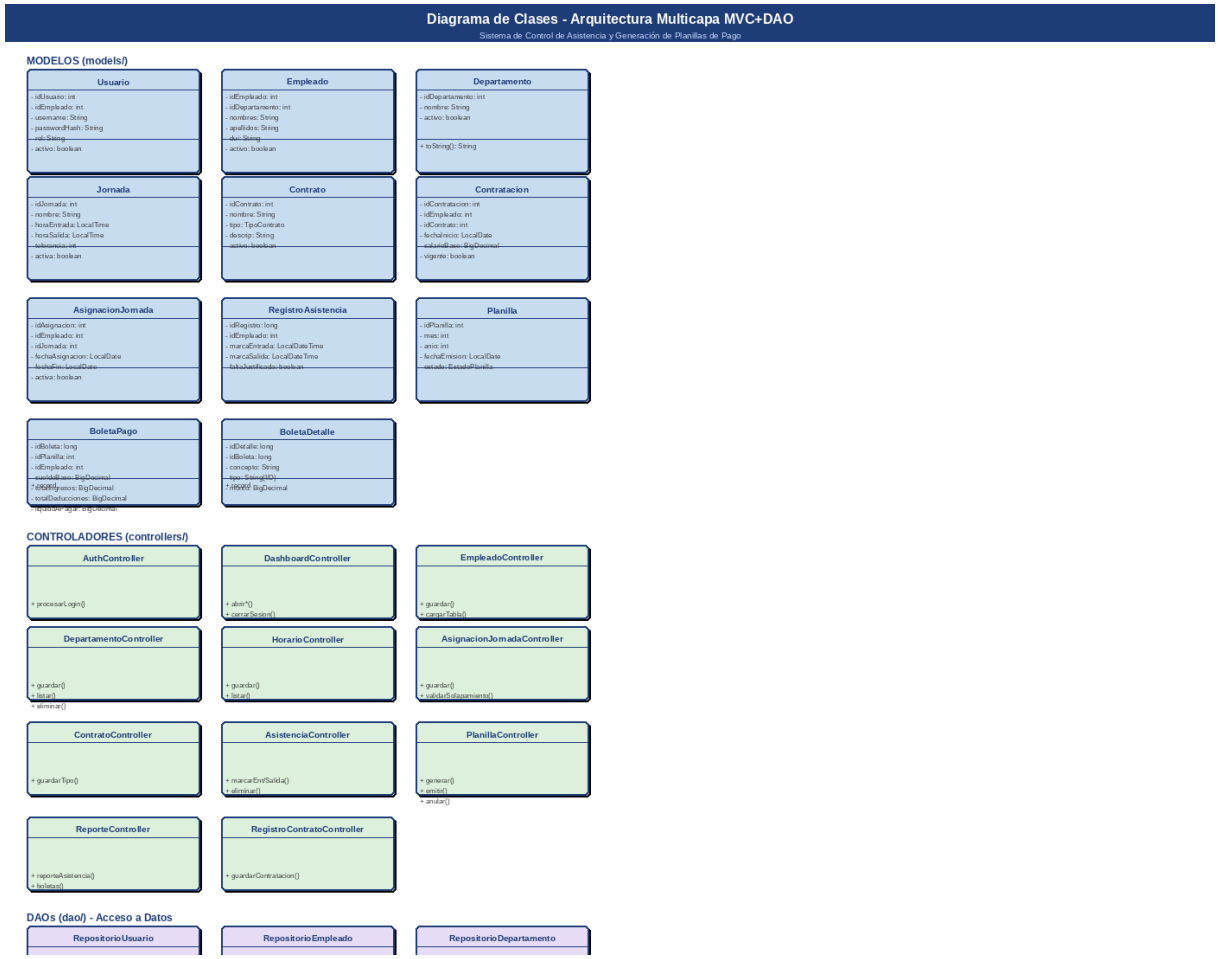
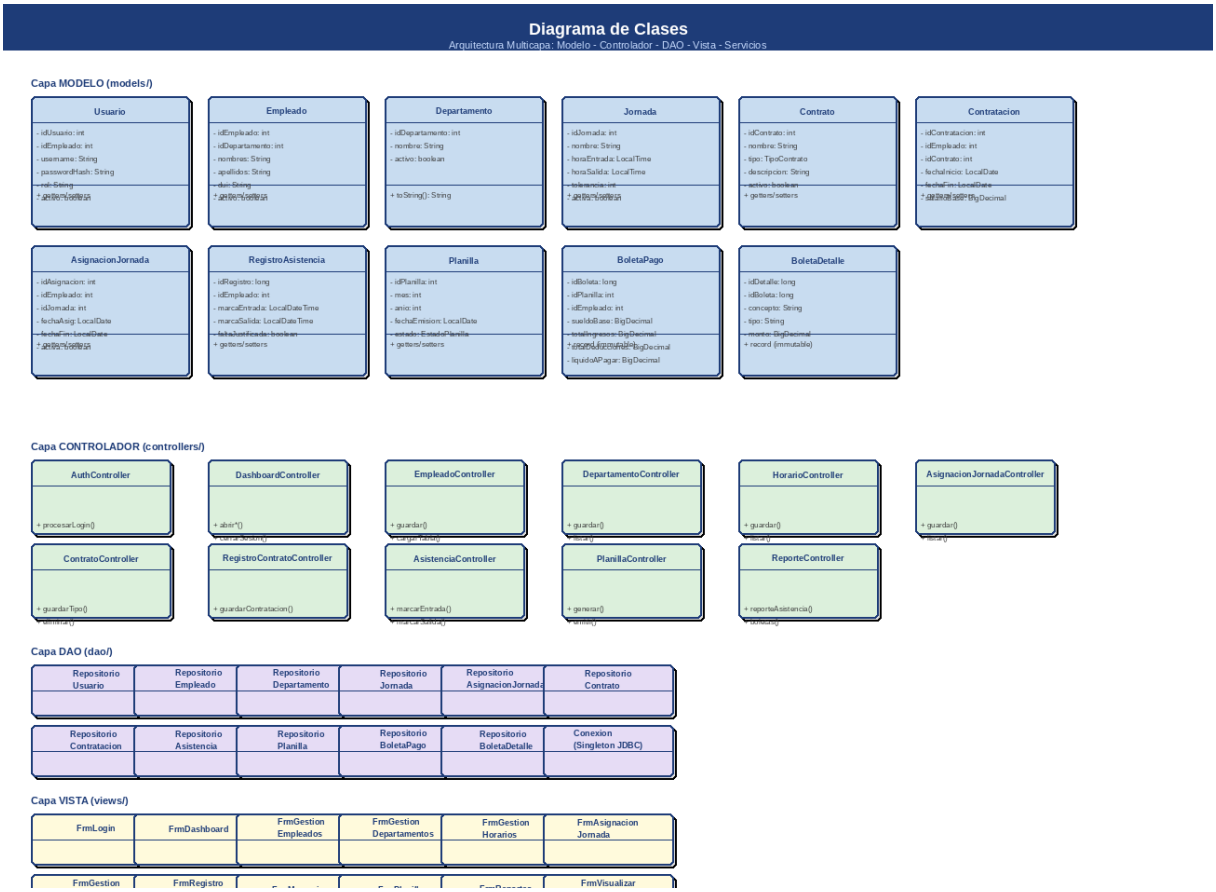


Figura 3.2: Diagrama de Clases - Arquitectura Multicapa (rediseñado)



3.1.3. Diagrama de Actividades

El diagrama de actividades describe el flujo de trabajo del proceso principal del sistema: el registro de asistencia y la generación de planillas. A continuación se describe el flujo de la actividad más crítica del sistema:

Flujo de Generación de Planilla:

1. El usuario inicia sesión en el sistema con sus credenciales.
2. Navega al módulo de Planillas desde el Dashboard.
3. Selecciona el mes y año para el cual desea generar la planilla.
4. El sistema verifica que no exista una planilla duplicada para el mismo mes/año.
5. El sistema verifica que todos los empleados activos tengan un contrato vigente.
6. Si las validaciones son exitosas, el sistema crea la planilla en estado 'Borrador'.
7. Para cada empleado activo con contrato vigente, el sistema genera una Boleta de Pago:
   a. Consulta los registros de asistencia del mes.
   b. Calcula las horas ordinarias trabajadas.
   c. Aplica el salario base o valor por hora según el tipo de contrato.
   d. Calcula las deducciones de ley: AFP (7.25%), ISSS (3%, tope $30), ISR (tabulado).
   e. Calcula el líquido a pagar (Total Ingresos - Total Deducciones).
   f. Persiste la boleta y sus detalles en la base de datos.
8. La planilla se marca como generada y puede ser emitida o anulada.
9. El usuario puede visualizar cada boleta individualmente o exportar reportes a CSV.

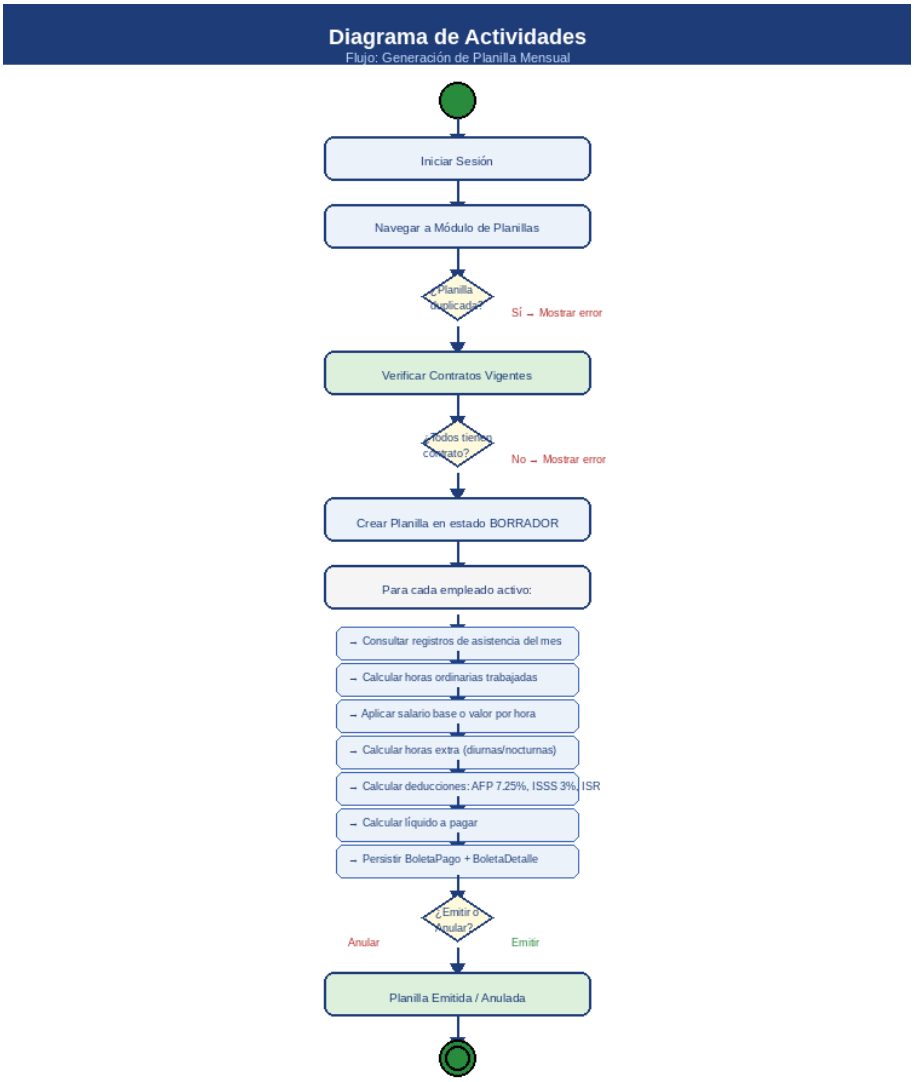
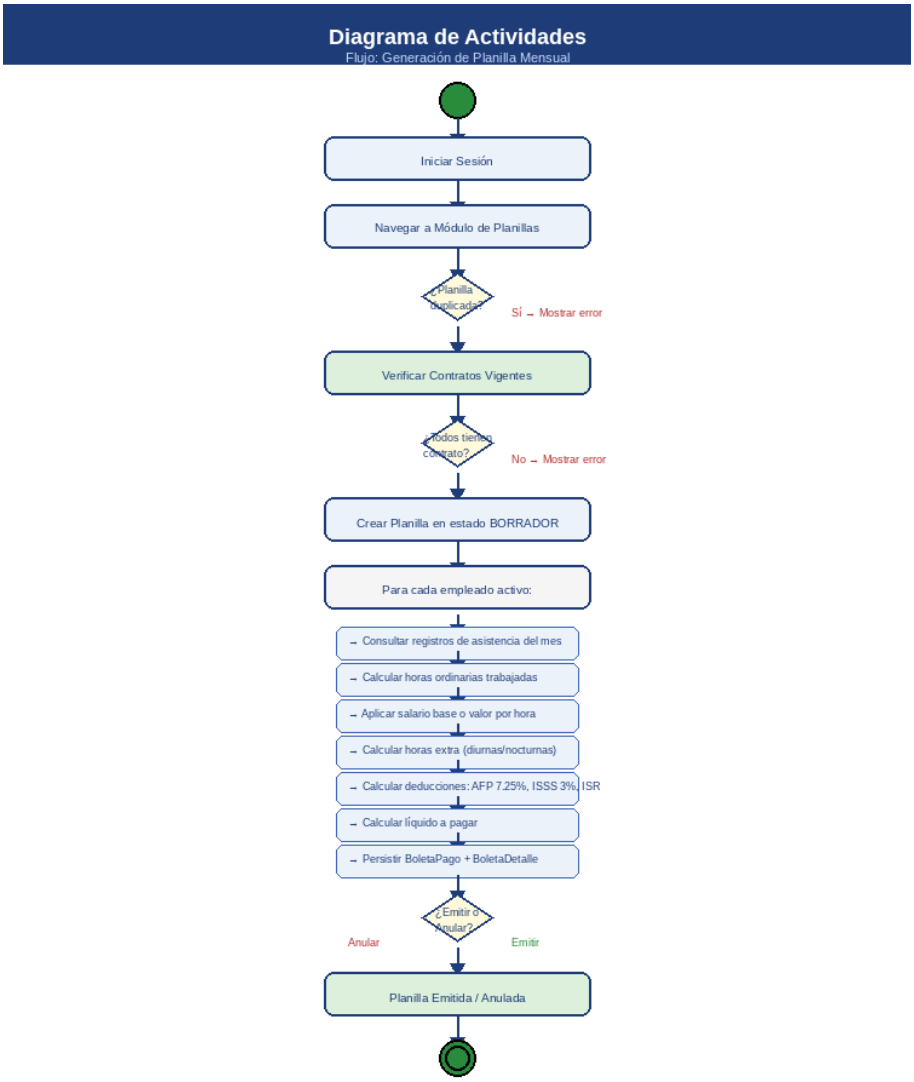


Figura 3.3: Diagrama de Actividades - Generación de Planilla



3.1.4. Diagrama de Secuencia

El diagrama de secuencia ilustra la interacción temporal entre los objetos del sistema durante el proceso de marcación de asistencia (entrada):

1. El usuario ingresa el código del empleado en FrmMarcacion y presiona 'Verificar'.
2. FrmMarcacion envía el idEmpleado a AsistenciaController.
3. AsistenciaController llama a RepositorioEmpleado.findById() para verificar que el empleado exista.
4. El controlador valida que el empleado esté activo.
5. AsistenciaController llama a tieneEntradaActiva() para verificar que no haya una marcación abierta.

6. Si todo es válido, el usuario presiona 'Marcar Entrada'.
7. AsistenciaController llama a marcarEntrada(idEmpleado).
8. El controlador invoca RepositorioAsistencia.registrar() con la fecha/hora actual.
9. El DAO ejecuta INSERT en la tabla RegistroAsistencia y obtiene el ID generado.
10. El controlador retorna un mensaje de éxito que se muestra en FrmMarcacion.
11. La tabla de marcaciones del día se actualiza automáticamente.

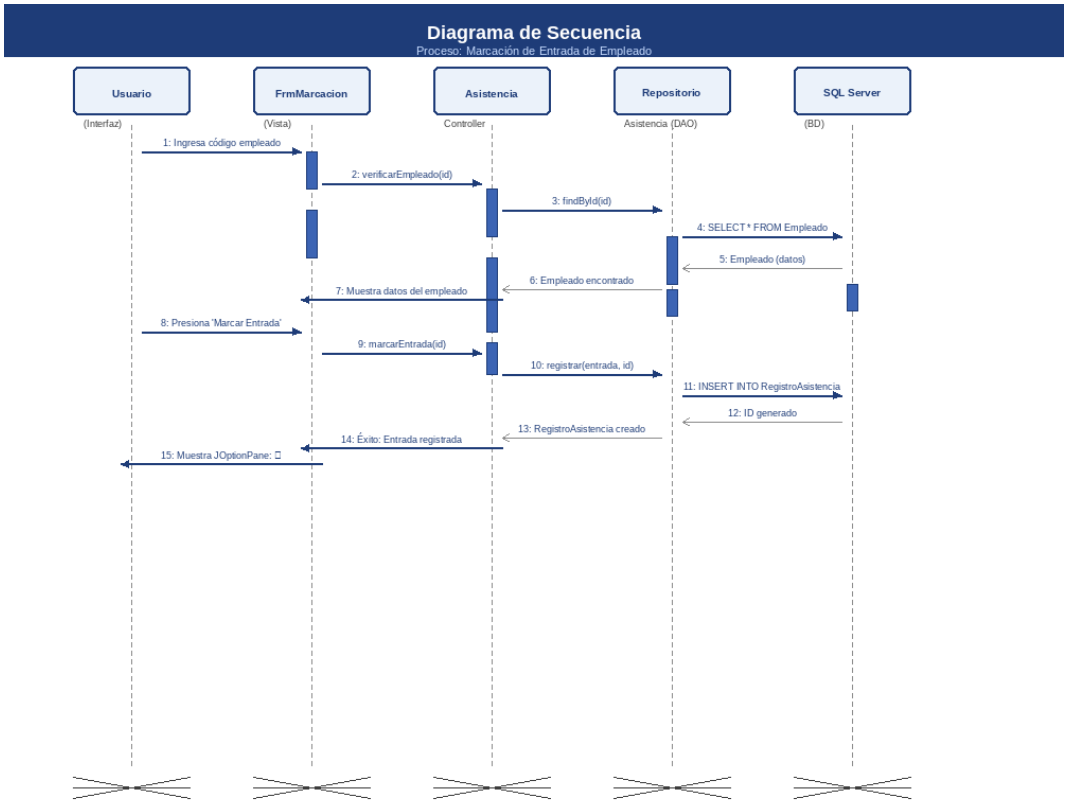
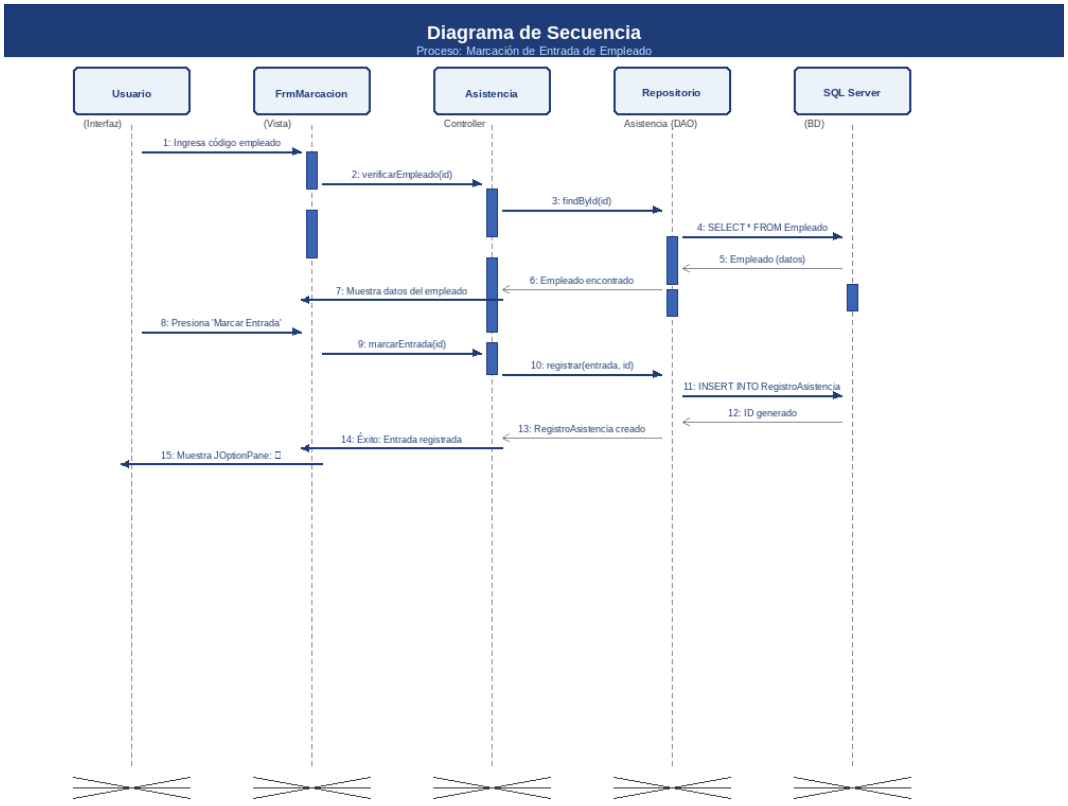


Figura 3.4: Diagrama de Secuencia - Marcación de Entrada



3.1.5. Diagrama de Estados

El diagrama de estados aplica principalmente a la entidad Planilla, que tiene un ciclo de vida claramente definido:

• BORRADOR → Estado inicial al generar la planilla. Permite visualizar boletas pero no realizar modificaciones. Es el único estado desde el cual se puede emitir.

• EMITIDA → La planilla ha sido oficializada. Las boletas ya no pueden modificarse ni anularse individualmente. Representa un documento contable válido.

• ANULADA → La planilla se ha cancelado. Solo puede anularse desde estado BORRADOR. Una vez anulada, no puede emitirse ni modificarse.

Las transiciones válidas son: BORRADOR → EMITIDA, BORRADOR → ANULADA. No se permite la transición EMITIDA → ANULADA por tratarse de un documento contable ya oficializado.

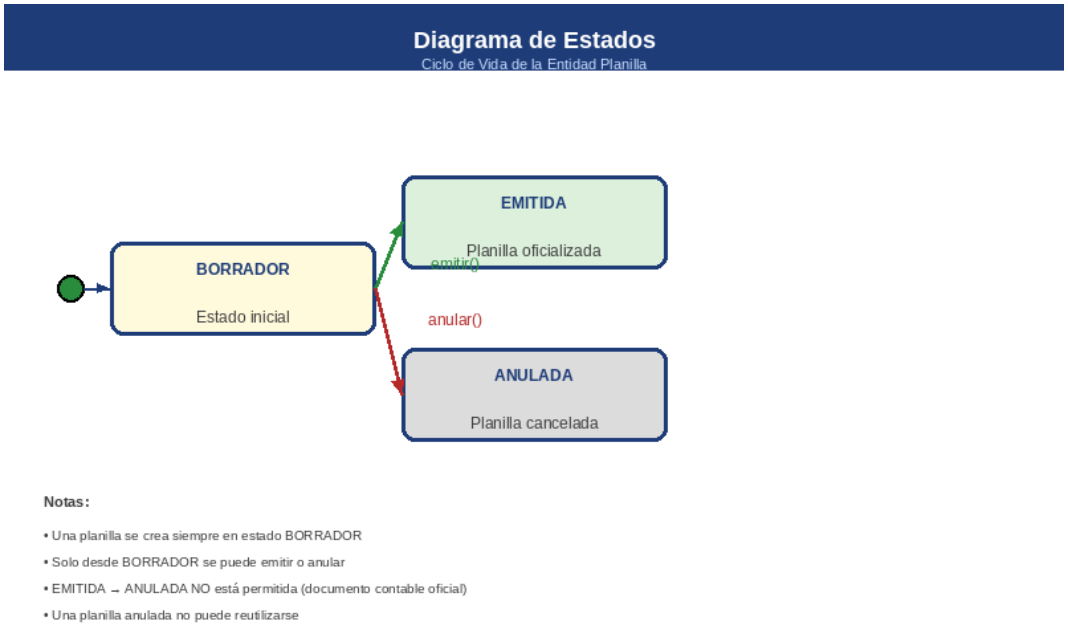
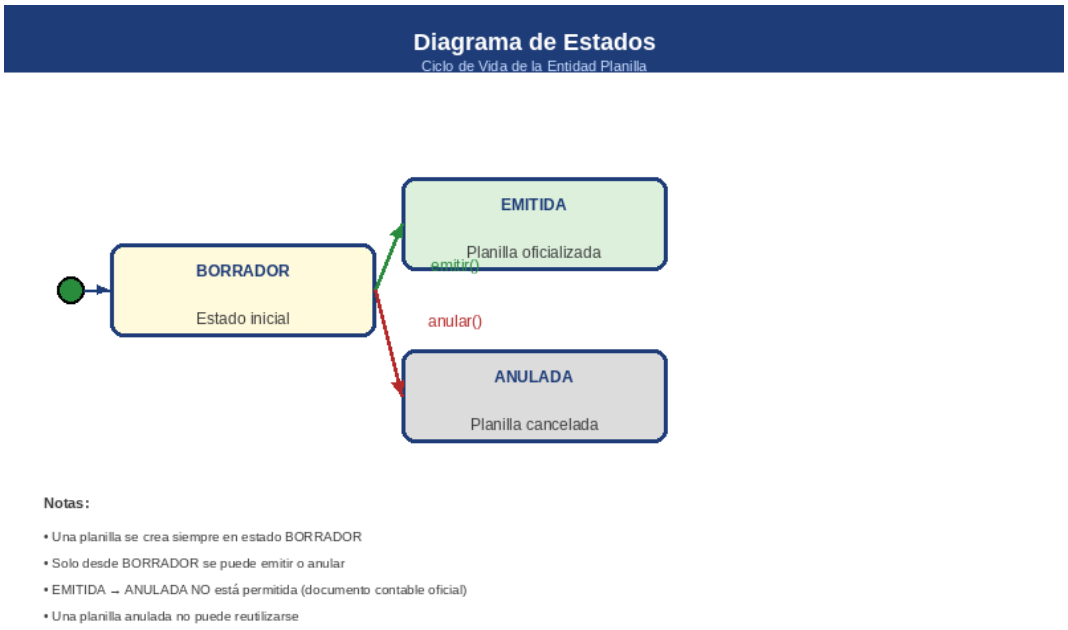


Figura 3.5: Diagrama de Estados - Ciclo de Vida de Planilla


Diagrama de Estados
Ciclo de Vida de la Entidad Planilla
EMITIDA
Planilla oficializada
emitir()
BORRADOR
Estado inicial
anular()
ANULADA
Planilla cancelada
Notas:
• Una planilla se crea siempre en estado BORRADOR
• Solo desde BORRADOR se puede emitir o anular
• EMITIDA → ANULADA NO está permitida (documento contable oficial)
• Una planilla anulada no puede reutilizarse

3.2. Mapa del Sistema

El sistema está organizado en módulos funcionales que se integran a través del Dashboard principal. A continuación se presenta el mapa gráfico del sistema:

Nivel	Componente	Descripción
Entrada	FrmLogin	Pantalla de autenticación de usuarios

Navegación	FrmDashboard	Menú principal con acceso a todos los módulos
Módulo 1	FrmGestionEmpleados	CRUD de empleados con asignación de departamento
Módulo 2	FrmGestionDepartamentos	Administración de departamentos
Módulo 3	FrmGestionHorarios	Definición de jornadas laborales
Módulo 4	FrmAsignacionJornada	Asignación de horarios a empleados
Módulo 5	FrmGestionContratos	Catálogo de tipos de contrato
Módulo 6	FrmRegistroContratos	Registro de contratación de empleados
Módulo 7	FrmMarcacion	Control de entrada/salida de empleados
Módulo 8	FrmPlanillas	Generación y gestión de planillas mensuales
Módulo 9	FrmReportes	Reportes de asistencia y planillas con exportación CSV
Módulo 10	FrmVisualizarBoleta	Detalle completo de boleta de pago individual
Servicios	GeneradorBoletas	Orquestación de generación de boletas
Servicios	CalculadoraHorasExtras	Cálculo de horas extra con factores legales
Servicios	CalculadoraDeduccionesLey	Cálculo de deducciones AFP, ISSS, ISR
Datos	Conexion + 11 DAOs	Capa de acceso a datos SQL Server
Salida	ExportadorCSV	Exportación de reportes a CSV con UTF-8 BOM

La arquitectura del sistema sigue el patrón MVC (Modelo-Vista-Controlador) complementado con el patrón DAO (Data Access Object). El flujo de datos es: Vista → Controlador → DAO → Base de Datos → DAO → Controlador → Vista. Los servicios de cálculo (GeneradorBoletas, CalculadoraHorasExtras, CalculadoraDeduccionesLey) son invocados por los controladores cuando se requiere lógica de negocio especializada.

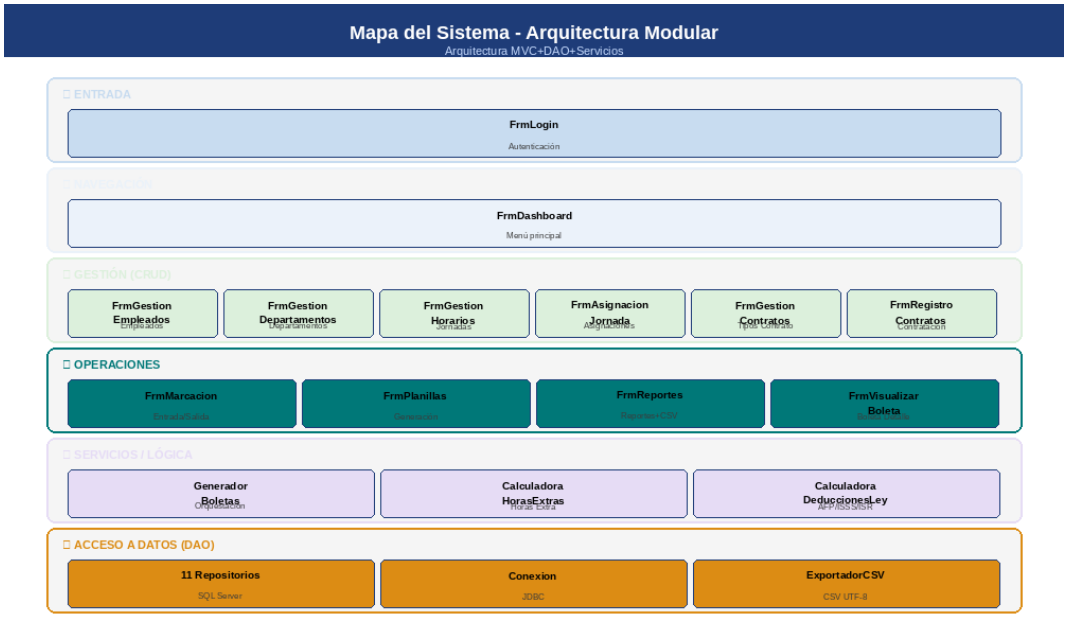


Figura 3.6: Mapa del Sistema - Arquitectura Modular

Mapa del Sistema - Arquitectura Modular
Arquitectura MVC+DAO+Servicios
ENTRADA
FrmLogin
Autenticación
NAVEGACIÓN
FrmDashboard
Menú principal
GESTIÓN (CRUD)
FrmGestion Empleados
FrmGestion Departamentos
FrmGestion Horarios
FrmAsignacion Jornada
FrmGestion Contratos
FrmRegistro Contratos
OPERACIONES
FrmMarcacion
FrmPlanillas
FrmReportes
FrmVisualizar Boleta
SERVICIOS / LÓGICA
Generador Boletas
Calculadora HorasExtras
Calculadora DeduccionesLey
ACCESO A DATOS (DAO)
11 Repositorios
SQL Server
Conexion
JDBC
ExportadorCSV
CSV UTF-8

3.3. Descripción de Módulos, Formularios y Clases

A continuación se describen detalladamente los módulos que componen el sistema, los formularios asociados y las clases principales de cada uno.

Módulo de Autenticación

Formulario: FrmLogin
Controlador: AuthController
DAO: RepositorioUsuario
Modelo: Usuario
Descripción: Permite a los usuarios iniciar sesión en el sistema mediante credenciales (usuario y contraseña). Incluye funcionalidad de mostrar/ocultar contraseña. Una vez autenticado, se inicia una sesión global (SesionGlobal) que mantiene el usuario activo durante toda la interacción con el sistema. El login valida que el usuario esté activo en la base de datos.

Módulo de Dashboard y Navegación

Formulario: FrmDashboard
Controlador: DashboardController
Descripción: Pantalla principal del sistema que presenta un menú con acceso a todos los módulos. Incluye atajos de teclado (Ctrl+E: Empleados, Ctrl+M: Marcación, Ctrl+P: Planillas, Ctrl+H: Horarios, Ctrl+C: Contratos, Ctrl+Q: Cerrar Sesión). Muestra un mensaje de bienvenida personalizado con el nombre del usuario que inició sesión.

Módulo de Gestión de Departamentos

Formulario: FrmGestionDepartamentos
Controlador: DepartamentoController
DAO: RepositorioDepartamento
Modelo: Departamento
Descripción: CRUD completo para administrar los departamentos de la empresa. Incluye validación de nombre duplicado, eliminación lógica (campo Activo) y listado en tabla.

Módulo de Gestión de Empleados

Formulario: FrmGestionEmpleados
Controlador: EmpleadoController
DAO: RepositorioEmpleado
Modelo: Empleado
Descripción: CRUD completo para el registro y mantenimiento de empleados. Los campos incluyen nombres, apellidos, DUI y departamento. Incluye validación de formato de DUI (########-#), verificación de DUI duplicado, selección de departamento mediante combo box y eliminación lógica. Permite seleccionar una fila de la tabla para editar los datos del empleado.

Módulo de Gestión de Horarios (Jornadas)

Formulario: FrmGestionHorarios
Controlador: HorarioController
DAO: RepositorioJornada
Modelo: Jornada
Descripción: CRUD para definir las jornadas laborales. Cada jornada tiene nombre, hora de entrada esperada, hora de salida esperada y margen de tolerancia en minutos. Las jornadas se gestionan mediante JSpinners personalizados sin modelo en el .form.

Módulo de Asignación de Jornadas

Formulario: FrmAsignacionJornada
Controlador: AsignacionJornadaController
DAO: RepositorioAsignacionJornada
Modelo: AsignacionJornada
Descripción: Permite asignar jornadas laborales a empleados con fechas de inicio y fin. Incluye validación de solapamiento de periodos usando la función COALESCE de SQL Server para manejar fechas fin nulas (asignaciones vigentes). Usa combos para seleccionar empleados y jornadas, y JSpinners para las fechas.

Módulo de Gestión de Contratos

Formulario: FrmGestionContratos
Controlador: ContratoController
DAO: RepositorioContrato
Modelo: Contrato
Descripción: Administración del catálogo de tipos de contrato. Los tipos disponibles son 'Fijo' y 'Por Hora'. Cada contrato tiene nombre, tipo, descripción y estado activo. Incluye validación de nombre duplicado y eliminación lógica.

Módulo de Registro de Contratación

Formulario: FrmRegistroContratos
Controlador: RegistroContratoController
DAO: RepositorioContratacion
Modelo: Contratacion
Descripción: Registra la vinculación contractual de un empleado con un tipo de contrato específico. Incluye selección de empleado y tipo de contrato, fechas de inicio y fin, salario base acordado y valor por hora. Para contratos 'Fijo' se requiere salario base; para 'Por Hora' se requiere valor por hora. El valor hora se calcula automáticamente como salario/160 cuando se ingresa un salario base.

Módulo de Marcación de Asistencia

Formulario: FrmMarcacion
Controlador: AsistenciaController
DAO: RepositorioAsistencia
Modelo: RegistroAsistencia
Descripción: Control de entrada y salida de empleados. El empleado ingresa su código y el sistema verifica su existencia y estado activo. Permite marcar entrada y salida con fecha/hora personalizable mediante JSpinners. Muestra la lista de marcaciones del día seleccionado con opción de eliminar registros. Calcula las horas trabajadas entre la entrada y la salida.

Módulo de Generación de Planillas

Formulario: FrmPlanillas
Controlador: PlanillaController
DAO: RepositorioPlanilla, RepositorioBoletaPago, RepositorioBoletaDetalle
Modelo: Planilla, BoletaPago, BoletaDetalle
Servicios: GeneradorBoletas, CalculadoraHorasExtras, CalculadoraDeduccionesLey
Descripción: Genera planillas mensuales con todas las boletas de pago de los empleados activos con contrato vigente. Utiliza transacciones SQL (commit/rollback) para asegurar la atomicidad de la operación. El cálculo de cada boleta incluye: sueldo base, horas ordinarias, horas extra (diurnas/nocturnas) con factores 2.0x y 2.25x, deducciones de AFP (7.25%), ISSS (3%, tope $30.00) e ISR (tabulado progresivo). Estados: Borrador, Emitida, Anulada.

Módulo de Reportes

Formulario: FrmReportes, FrmVisualizarBoleta
Controlador: ReporteController
DAO: RepositorioAsistencia, RepositorioBoletaPago
Utilidad: ExportadorCSV
Descripción: Genera reportes de asistencia por empleado y rango de fechas, y reportes de planilla con todas las boletas emitidas. Permite exportar los reportes a formato CSV con codificación UTF-8 BOM para compatibilidad con Excel. Incluye vista detallada de boleta individual (FrmVisualizarBoleta) con desglose de ingresos, deducciones y líquido a pagar.

3.4. Descripción de Funciones

El sistema implementa las siguientes funciones principales, organizadas por módulo:

Módulo	Función	Descripción
Autenticación	procesarLogin()	Valida credenciales del usuario contra la BD
Autenticación	cerrarSesion()	Finaliza la sesión y retorna al login
Empleados	guardarEmpleado()	Crea o actualiza un empleado con validación de DUI
Empleados	desactivarEmpleado()	Eliminación lógica de un empleado
Empleados	cargarTabla()	Carga el listado de empleados en la tabla
Departamentos	guardar()	Crea un departamento con validación de duplicado
Departamentos	actualizar()	Actualiza datos de un departamento
Departamentos	eliminar()	Desactiva un departamento
Horarios	guardar(Jornada)	Crea una jornada con validación de horarios
Horarios	actualizar(Jornada)	Actualiza una jornada existente
Horarios	eliminar(int)	Desactiva una jornada
Asig. Jornada	guardar()	Asigna jornada a empleado con validación de solapamiento
Asig. Jornada	actualizar()	Actualiza una asignación existente
Asig. Jornada	eliminar()	Eliminación lógica de asignación
Asig. Jornada	listarConNombres()	Lista asignaciones con nombres de empleado y jornada
Contratos	guardarTipoContrato()	Crea un tipo de contrato (Fijo/Por Hora)
Contratos	eliminarTipoContrato()	Elimina un tipo de contrato
Contratación	guardarContratacion()	Registra contratación de empleado con fechas y salario
Asistencia	marcarEntrada()	Registra entrada de empleado con fecha/hora
Asistencia	marcarSalida()	Registra salida del empleado
Asistencia	eliminarAsistencia()	Elimina un registro de asistencia
Asistencia	calcularHorasTrabajadas()	Calcula horas entre entrada y salida
Planillas	generarNuevaPlanilla()	Genera planilla mensual con todas

		las boletas
Planillas	emitirPlanilla()	Cambia estado de planilla a Emitida
Planillas	anularPlanilla()	Cambia estado de planilla a Anulada
Planillas	filtrarPorEstado()	Filtra planillas por su estado
Reportes	reporteAsistencia()	Genera reporte de asistencia por empleado y rango
Reportes	boletasPorPlanilla()	Lista boletas de una planilla específica
Reportes	boletaCompleta()	Obtiene detalle completo de una boleta de pago
Reportes	totalesPlanilla()	Calcula totales agregados de una planilla
Exportación	exportarJTable()	Exporta cualquier JTable a CSV UTF-8 BOM
Servicios	generarBoletas()	Orquesta generación de boletas para todos los empleados
Servicios	calcularSalario()	Calcula salario según tipo de contrato
Servicios	calcularDeducciones()	Calcula AFP, ISSS e ISR según ley salvadoreña

3.5. Base de Datos

La base de datos del sistema se implementó en Microsoft SQL Server, utilizando el motor de base de datos relacional para garantizar la integridad, consistencia y seguridad de la información. A continuación se presentan el modelo entidad-relación y el diccionario de datos.

3.5.1. Modelo Entidad-Relación

El modelo entidad-relación (DER) del sistema está compuesto por 11 tablas que representan las entidades del dominio y sus relaciones:

Tabla	Tipo	Descripción
Departamento	Maestra	Almacena los departamentos de la empresa
Empleado	Maestra	Datos personales y laborales de los empleados
Usuario	Maestra	Credenciales de acceso al sistema
Contrato	Catálogo	Tipos de contrato disponibles (Fijo/Por Hora)
Contratacion	Transaccional	Vinculación empleado-contrato con fechas y salario
Jornada	Catálogo	Definición de jornadas laborales con horarios
AsignacionJornada	Transaccional	Asignación de jornadas a empleados por periodo
RegistroAsistencia	Transaccional	Registro de marcaciones de

		entrada/salida
Planilla	Transaccional	Cabecera de planilla mensual
BoletaPago	Transaccional	Boleta individual de pago por empleado
BoletaDetalle	Transaccional	Detalle de ingresos y deducciones de cada boleta

Relaciones entre tablas:

• Departamento (1) → (N) Empleado: Un departamento puede tener muchos empleados.
• Empleado (1) → (1) Usuario: Cada empleado tiene un único usuario de sistema.
• Empleado (1) → (N) Contratacion: Un empleado puede tener múltiples contrataciones en el tiempo.
• Contrato (1) → (N) Contratacion: Un tipo de contrato puede aplicarse a múltiples contrataciones.
• Empleado (1) → (N) AsignacionJornada: Un empleado puede tener múltiples asignaciones de jornada.
• Jornada (1) → (N) AsignacionJornada: Una jornada puede asignarse a múltiples empleados.
• Empleado (1) → (N) RegistroAsistencia: Un empleado genera múltiples registros de asistencia.
• Empleado (1) → (N) BoletaPago: Un empleado recibe una boleta por cada planilla del periodo.
• Planilla (1) → (N) BoletaPago: Una planilla contiene boletas para todos los empleados.
• BoletaPago (1) → (N) BoletaDetalle: Cada boleta tiene múltiples líneas de detalle.

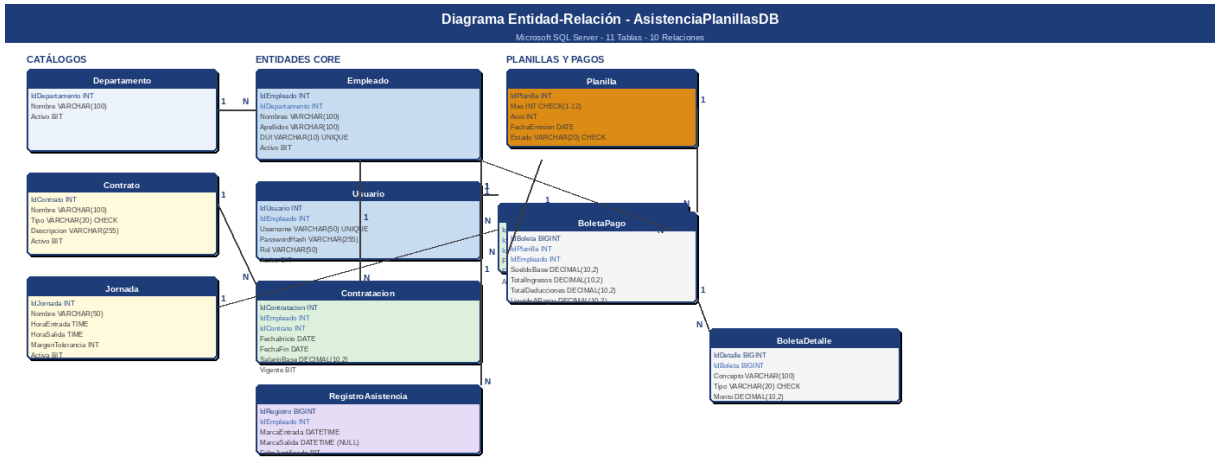
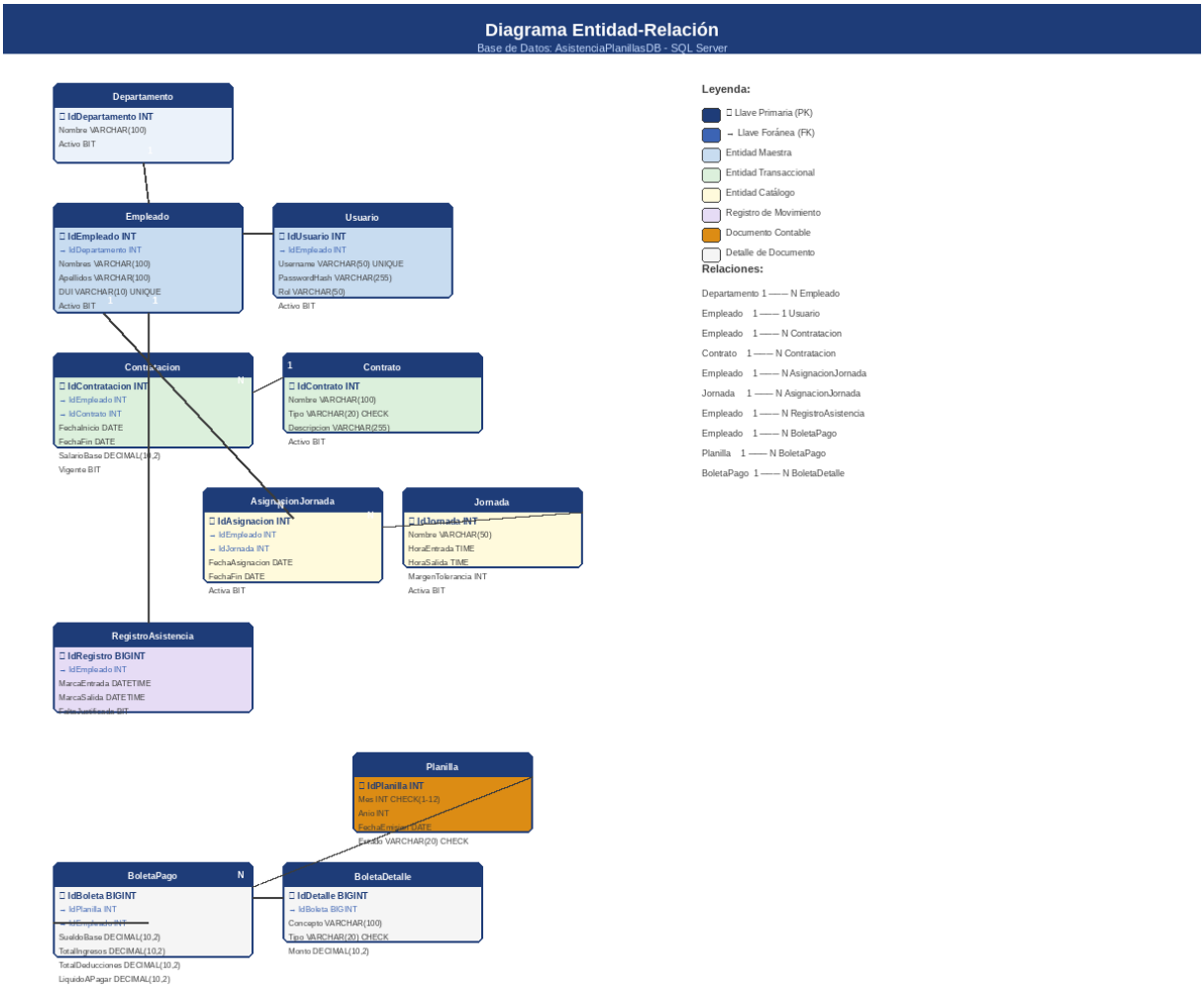


Figura 3.7: Diagrama Entidad-Relación - AsistenciaPlanillasDB (rediseñado)



3.5.2. Diccionario de Datos

A continuación se presenta el diccionario de datos de cada tabla del sistema:

Tabla: Departamento

Campo	Tipo	Restricciones	Descripción
IdDepartamento	INT	IDENTITY(1,1) PK	Identificador único del departamento
Nombre	VARCHAR(100)	NOT NULL	Nombre del departamento
Activo	BIT	NOT NULL DEFAULT 1	Indica si el departamento está activo

Tabla: Empleado

Campo	Tipo	Restricciones	Descripción
IdEmpleado	INT	IDENTITY(1,1) PK	Identificador único del empleado
IdDepartamento	INT	FK → Departamento(IdDepartamento)	Departamento al que pertenece

Nombres	VARCHAR(100)	NOT NULL	Nombres del empleado
Apellidos	VARCHAR(100)	NOT NULL	Apellidos del empleado
DUI	VARCHAR(10)	NOT NULL UNIQUE	Documento Único de Identidad
Activo	BIT	NOT NULL DEFAULT 1	Indica si el empleado está activo

Tabla: Usuario

Campo	Tipo	Restricciones	Descripción
IdUsuario	INT	IDENTITY(1,1) PK	Identificador único del usuario
IdEmpleado	INT	FK → Empleado(IdEmpleado)	Empleado asociado al usuario
Username	VARCHAR(50)	NOT NULL UNIQUE	Nombre de usuario para inicio de sesión
PasswordHash	VARCHAR(255)	NOT NULL	Contraseña del usuario
Rol	VARCHAR(50)	NOT NULL DEFAULT 'RRHH'	Rol del usuario: Admin, RRHH, Empleado
Activo	BIT	NOT NULL DEFAULT 1	Indica si el usuario está activo

Tabla: Contrato

Campo	Tipo	Restricciones	Descripción
IdContrato	INT	IDENTITY(1,1) PK	Identificador único del tipo de contrato
Nombre	VARCHAR(100)	NOT NULL	Nombre del tipo de contrato
Tipo	VARCHAR(20)	NOT NULL CHECK('Fijo','Por Hora')	Categoría del contrato
Descripcion	VARCHAR(255)	NULL	Descripción del tipo de contrato
Activo	BIT	NOT NULL DEFAULT 1	Indica si el tipo de contrato está activo

Tabla: Contratacion

Campo	Tipo	Restricciones	Descripción
IdContratacion	INT	IDENTITY(1,1) PK	Identificador único de la contratación
IdEmpleado	INT	FK → Empleado(IdEmpleado)	Empleado contratado
IdContrato	INT	FK → Contrato(IdContrato)	Tipo de contrato aplicado
FechaInicio	DATE	NOT NULL	Fecha de inicio de la contratación
FechaFin	DATE	NULL	Fecha de fin (NULL = vigente)
SalarioBaseAcordado	DECIMAL(10,2)	CHECK(>=0)	Salario base mensual

ValorHoraAcordado	DECIMAL(10,2)	CHECK(>=0)	Valor por hora
Vigente	BIT	NOT NULL DEFAULT 1	Indica si la contratación está vigente

Tabla: Jornada

Campo	Tipo	Restricciones	Descripción
IdJornada	INT	IDENTITY(1,1) PK	Identificador único de la jornada
Nombre	VARCHAR(50)	NOT NULL	Nombre de la jornada
HoraEntradaEsperada	TIME	NOT NULL	Hora esperada de entrada
HoraSalidaEsperada	TIME	NOT NULL	Hora esperada de salida
MargenToleranciaMinutos	INT	NOT NULL DEFAULT 0	Minutos de tolerancia
Activa	BIT	NOT NULL DEFAULT 1	Indica si la jornada está activa

Tabla: AsignacionJornada

Campo	Tipo	Restricciones	Descripción
IdAsignacion	INT	IDENTITY(1,1) PK	Identificador único de la asignación
IdEmpleado	INT	FK → Empleado(IdEmpleado)	Empleado asignado
IdJornada	INT	FK → Jornada(IdJornada)	Jornada asignada
FechaAsignacion	DATE	NOT NULL DEFAULT GETDATE()	Fecha de asignación
FechaFin	DATE	NULL	Fecha de fin de asignación
Activa	BIT	NOT NULL DEFAULT 1	Indica si la asignación está activa

Tabla: RegistroAsistencia

Campo	Tipo	Restricciones	Descripción
IdRegistro	BIGINT	IDENTITY(1,1) PK	Identificador único del registro
IdEmpleado	INT	FK → Empleado(IdEmpleado)	Empleado que registra asistencia
MarcaEntrada	DATETIME	NOT NULL	Fecha y hora de entrada
MarcaSalida	DATETIME	NULL	Fecha y hora de salida
FaltaJustificada	BIT	NOT NULL DEFAULT 0	Indica si es una falta justificada

Tabla: Planilla

Campo	Tipo	Restricciones	Descripción
IdPlanilla	INT	IDENTITY(1,1) PK	Identificador único de la planilla
Mes	INT	NOT NULL CHECK(1-12)	Mes de la planilla
Anio	INT	NOT NULL	Año de la planilla

FechaEmision	DATE	NOT NULL DEFAULT GETDATE()	Fecha de emisión
Estado	VARCHAR(20)	NOT NULL DEFAULT 'Borrador' CHECK('Borrador','Emitida','Anulada')	Estado de la planilla

Tabla: BoletaPago

Campo	Tipo	Restricciones	Descripción
IdBoleta	BIGINT	IDENTITY(1,1) PK	Identificador único de la boleta
IdPlanilla	INT	FK → Planilla(IdPlanilla)	Planilla a la que pertenece
IdEmpleado	INT	FK → Empleado(IdEmpleado)	Empleado beneficiario
SueldoBaseAplicado	DECIMAL(10,2)	NOT NULL	Salario base del periodo
HorasOrdinariasCalculadas	DECIMAL(5,2)	NOT NULL DEFAULT 0	Horas ordinarias trabajadas
HorasExtraDiurnas	DECIMAL(5,2)	NOT NULL DEFAULT 0	Horas extra diurnas
HorasExtraNocturnas	DECIMAL(5,2)	NOT NULL DEFAULT 0	Horas extra nocturnas
TotalIngresos	DECIMAL(10,2)	NOT NULL DEFAULT 0	Total de ingresos del periodo
TotalDeducciones	DECIMAL(10,2)	NOT NULL DEFAULT 0	Total de deducciones
LiquidoAPagar	DECIMAL(10,2)	NOT NULL DEFAULT 0	Salario neto a pagar

Tabla: BoletaDetalle

Campo	Tipo	Restricciones	Descripción
IdDetalle	BIGINT	IDENTITY(1,1) PK	Identificador único del detalle
IdBoleta	BIGINT	FK → BoletaPago(IdBoleta)	Boleta a la que pertenece
Concepto	VARCHAR(100)	NOT NULL	Concepto del ingreso o deducción
Tipo	VARCHAR(20)	NOT NULL CHECK('Ingreso','Deduccion')	Tipo: Ingreso o Deducción
Monto	DECIMAL(10,2)	NOT NULL	Monto del concepto

3.6. Seguridad del Sistema

La seguridad del sistema se aborda desde tres perspectivas: control de acceso, integridad de datos y seguridad a nivel de base de datos.

3.6.1. Control de Acceso (Claves)

El sistema implementa un mecanismo de autenticación mediante credenciales (usuario y contraseña). Cada usuario registrado en el sistema tiene un nombre de usuario único y una contraseña asociada. El proceso de autenticación verifica que:

• El nombre de usuario exista en la tabla Usuario.

• La contraseña proporcionada coincida con la almacenada.

• El usuario esté marcado como activo (Activo = 1) en la base de datos.

• Solo los usuarios autenticados pueden acceder al Dashboard y sus funcionalidades.

• La sesión se mantiene mediante la clase SesionGlobal, que almacena el objeto Usuario activo durante toda la interacción con el sistema.

• Al cerrar sesión, se destruye la referencia al usuario activo y se retorna a la pantalla de login, impidiendo el acceso no autorizado a las funcionalidades del sistema.

3.6.2. Roles y Privilegios

El sistema contempla tres roles de usuario con diferentes niveles de acceso:

Rol	Privilegios	Descripción
Admin	Todos los módulos	Acceso completo a todas las funcionalidades del sistema
RRHH	Gestión de empleados, asistencia, planillas, reportes	Acceso a operaciones de recursos humanos
Empleado	Marcación de asistencia (entrada/salida)	Acceso limitado solo a marcación

Actualmente, el sistema se encuentra en una fase donde todos los usuarios autenticados tienen acceso al Dashboard completo. La segmentación por roles basada en el campo 'Rol' de la tabla Usuario está prevista para la Fase 5 del proyecto, donde se implementará la restricción de visibilidad de menús y operaciones según el rol del usuario.

3.6.3. Seguridad a Nivel de Base de Datos

A nivel de base de datos, se implementaron las siguientes medidas de seguridad:

• Integridad Referencial: Todas las tablas transaccionales utilizan llaves foráneas (FK) que garantizan la integridad de las relaciones entre entidades. Por ejemplo, no se puede registrar una asistencia para un empleado que no exista en la tabla Empleado.

• Restricciones CHECK: Se implementaron restricciones CHECK en campos críticos: mes entre 1 y 12 en Planilla, salario base y valor hora >= 0 en Contratacion, tipo de contrato restringido a 'Fijo' o 'Por Hora', estado de planilla a 'Borrador', 'Emitida' o 'Anulada'.

• Valores por Defecto: Los campos Activo, Vigente y FaltaJustificada tienen valores por defecto que aseguran un estado inicial seguro sin requerir intervención manual.

• Campos UNIQUE: El DUI del empleado y el Username del usuario tienen restricciones UNIQUE que previenen duplicados.

• Transacciones: La generación de planillas se ejecuta dentro de una transacción SQL Server (BEGIN TRANSACTION / COMMIT / ROLLBACK) para asegurar que la creación de la planilla y todas sus boletas se realice de manera atómica: si falla una boleta, se revierte todo el proceso y no queda información inconsistente en la base de datos.

• Conexión Segura: La conexión JDBC utiliza autenticación de SQL Server con credenciales de aplicación (sa) y certificado de confianza para el entorno de desarrollo.

• Eliminación Lógica: Ninguna entidad se elimina físicamente de la base de datos. Todas las tablas maestras y transaccionales utilizan un campo Activo (BIT) que permite la desactivación lógica de registros, preservando la integridad histórica de los datos.

3.6.4. Buenas Prácticas de Programación

El código fuente del sistema sigue las siguientes buenas prácticas de seguridad y calidad:

• Arquitectura en Capas: La separación en Modelo-Vista-Controlador-DAO-Servicios asegura que la lógica de negocio y el acceso a datos estén aislados de la interfaz de usuario, facilitando el mantenimiento y la auditoría del código.

• Validación de Datos: Todos los formularios validan la entrada del usuario antes de enviarla a la base de datos. Por ejemplo, el formato del DUI se valida con expresión regular (\d{8}-\d), los campos numéricos se validan contra valores negativos, y las fechas se verifican para evitar inconsistencias temporales.

• Manejo de Excepciones: Los controladores capturan y manejan excepciones SQL y de validación, mostrando mensajes descriptivos al usuario mediante JOptionPane en lugar de imprimir errores en consola.

• Programación Funcional: El sistema utiliza características de programación funcional de Java 8+ como Optional, streams y lambdas para un código más legible y seguro.

• Principio de Mínimo Privilegio: Las consultas SQL utilizan exclusivamente los datos necesarios para cada operación, evitando la exposición de información sensible no requerida.

Fuentes de consulta

Pressman, R. S. (2010). Ingeniería de Software: Un Enfoque Práctico (7ma ed.). McGraw-Hill.

Girigan, M. G. (2025, 10 abril). Introduction to Modular Monoliths: Optimal Architecture - BoatyardX. BoatyardX. Recuperado el 22 de enero de 2026 de https://boatyardx.com/introduction-to-modular-monoliths-optimal-architecture/.

El Órgano Ejecutivo de la República de El Salvador. (s. f.). TABLAS DE RETENCION DEL IMPUESTO SOBRE LA RENTA. En Ministerio de Hacienda. Recuperado el 22 de enero de 2026 de https://transparencia.mh.gob.sv/downloads/pdf/DC5852_Tablas%20de%20retencion%20del%20ISR.pdf.

Schmuller, J. (2004). Aprendiendo UML en 24 Horas. Pearson Educación.

Kendall, K. E. y Kendall, J. E. (2011). Análisis y Diseño de Sistemas (8va ed.). Pearson.

Deitel, H. M. y Deitel, P. J. (2016). Java: Cómo Programar (10ma ed.). Pearson Educación.

Silberschatz, A., Korth, H. F. y Sudarshan, S. (2019). Fundamentos de Bases de Datos (7ma ed.). McGraw-Hill.

Sommerville, I. (2016). Software Engineering (10ma ed.). Pearson.

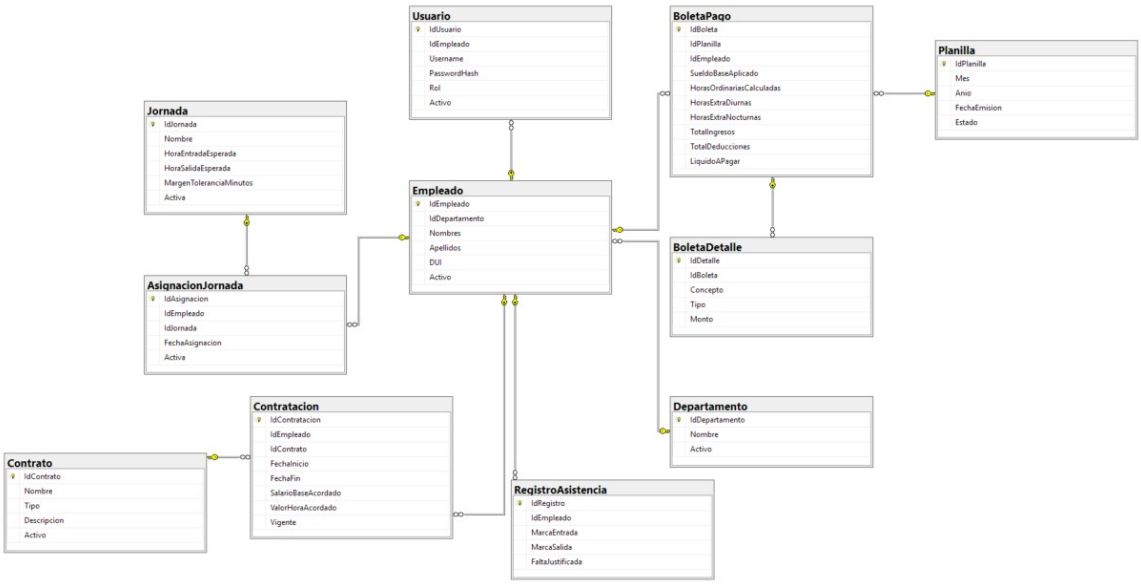
El Autor. (2026). Documentación Técnica del Sistema de Control de Asistencia y Generación de Planillas de Pago. [Documento interno del proyecto].

Glosario de términos

1. AFP: Administradora de Fondos de Pensiones. Institución financiera encargada de administrar las cuentas individuales de ahorro para pensiones.
2. DAO: Data Access Object. Patrón de diseño arquitectónico que separa la lógica de acceso a la base de datos de la lógica de negocios de la aplicación.
3. ISSS: Instituto Salvadoreño del Seguro Social. Entidad encargada de brindar atención médica y previsional a los trabajadores.
4. Planilla: Documento contable que refleja el cálculo detallado de los ingresos, retenciones y salario neto que percibe un empleado por su trabajo en un periodo determinado.

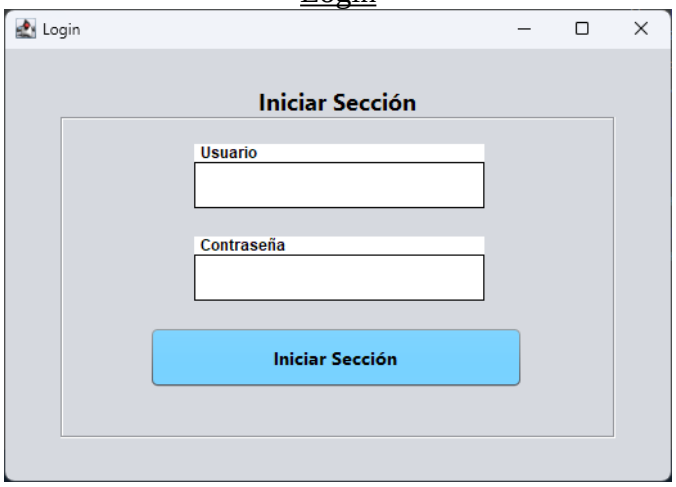
Anexos

Anexo A: Diseño del Modelo Lógico Relacional de la base de datos en SQL Server.

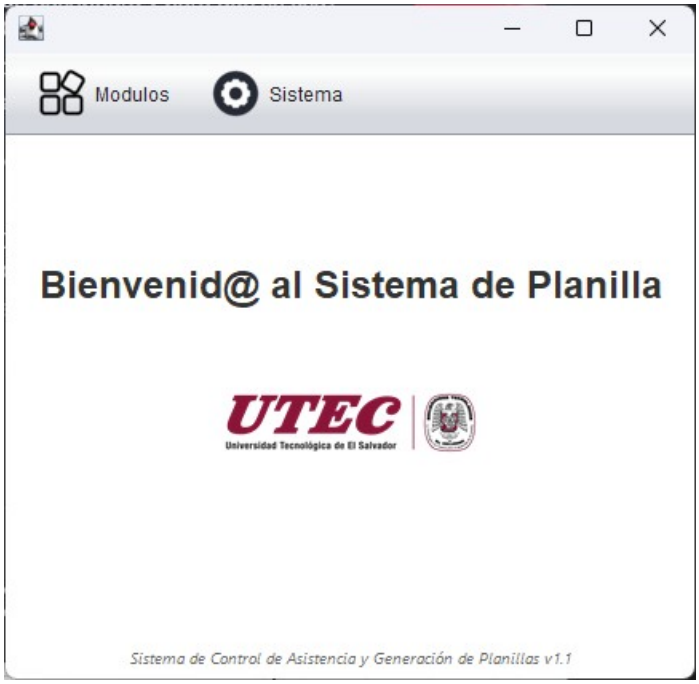


Anexo B: Capturas de pantalla del prototipo funcional en Java SE.

Login



Dashboard



Módulo de Empleados

